

NumPy Documentation: Array Creation

Source: <https://numpy.org/doc/stable/user/basics.creation.html>

Introduction

There are 6 general mechanisms for creating arrays:

1. Conversion from other Python structures (i.e. lists and tuples)
2. Intrinsic NumPy array creation functions (e.g. `arange`, `ones`, `zeros`, etc.)
3. Replicating, joining, or mutating existing arrays
4. Reading arrays from disk, either from standard or custom formats
5. Creating arrays from raw bytes through the use of strings or buffers
6. Use of special library functions (e.g., `random`)

1) Converting Python Sequences to NumPy Arrays

NumPy arrays can be defined using Python sequences such as lists and tuples.

- a list of numbers will create a 1D array,
- a list of lists will create a 2D array,
- further nested lists will create higher-dimensional arrays.

```
import numpy as np
a1D = np.array([1, 2, 3, 4])
a2D = np.array([[1, 2], [3, 4]])
a3D = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

When you use `numpy.array` to define a new array, you should consider the `dtype` of the elements, which can be specified explicitly. When values do not fit and you are using a `dtype`, NumPy may raise an error:

```
>>> np.array([127, 128, 129], dtype=np.int8)
OverflowError: Python integer 128 out of bounds for int8
```

If you perform calculations with mismatching dtypes, you can get unwanted results:

```
>>> a = np.array([2, 3, 4], dtype=np.uint32)
>>> b = np.array([5, 6, 7], dtype=np.uint32)
>>> c_unsigned32 = a - b
>>> print('unsigned c:', c_unsigned32, c_unsigned32.dtype)
unsigned c: [4294967293 4294967293 4294967293] uint32
>>> c_signed32 = a - b.astype(np.int32)
>>> print('signed c:', c_signed32, c_signed32.dtype)
signed c: [-3 -3 -3] int64
```

The default NumPy behavior is to create arrays in either 32 or 64-bit signed integers (platform dependent) or double precision floating point numbers. If you expect your integer arrays to be a specific type, specify the `dtype` while creating the array.

2) Intrinsic NumPy Array Creation Functions

NumPy has over 40 built-in functions for creating arrays, split into roughly three categories based on dimension.

1D array creation functions

`numpy.arange` and `numpy.linspace` generally need at least two inputs, start and stop.

`numpy.arange` creates arrays with regularly incrementing values:

```
>>> np.arange(10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.arange(2, 10, dtype=np.float64)
array([2., 3., 4., 5., 6., 7., 8., 9.])
>>> np.arange(2, 3, 0.1)
array([2. , 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9])
```

`numpy.linspace` creates arrays with a specified number of elements, spaced equally between the specified beginning and end values:

```
>>> np.linspace(1., 4., 6)
array([1. , 1.6, 2.2, 2.8, 3.4, 4. ])
```

The advantage of `linspace` is that you guarantee the number of elements and the starting and end point; `arange(start, stop, step)` will not include the value `stop`.

2D array creation functions

`numpy.eye`, `numpy.diag`, and `numpy.vander` define properties of special matrices represented as 2D arrays.

`np.eye(n, m)` defines a 2D identity matrix:

```
>>> np.eye(3)
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

`numpy.diag` can define either a square 2D array with given values along the diagonal, or, if given a 2D array, returns a 1D array that is only the diagonal elements:

```
>>> np.diag([1, 2, 3])
array([[1, 0, 0],
       [0, 2, 0],
       [0, 0, 3]])
>>> a = np.array([[1, 2], [3, 4]])
>>> np.diag(a)
array([1, 4])
```

General ndarray creation functions

`numpy.ones`, `numpy.zeros`, and `random` define arrays based upon the desired shape.

`numpy.zeros` creates an array filled with 0 values, default dtype `float64`:

```
>>> np.zeros((2, 3))
array([[0., 0., 0.],
       [0., 0., 0.]])
```

`numpy.ones` creates an array filled with 1 values:

```
>>> np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])
```

The `random` method of `default_rng` creates an array filled with random values between 0 and 1:

```
>>> from numpy.random import default_rng
>>> default_rng(42).random((2,3))
array([[0.77395605, 0.43887844, 0.85859792],
       [0.69736803, 0.09417735, 0.97562235]])
```

3) Replicating, Joining, or Mutating Existing Arrays

When you assign an array or its elements to a new variable, you have to explicitly `numpy.copy` the array, otherwise the variable is a view into the original array.

```
>>> a = np.array([1, 2, 3, 4, 5, 6])
>>> b = a[:2]
>>> b += 1
>>> print('a =', a, '; b =', b)
a = [2 3 3 4 5 6] ; b = [2 3]
```

To create a new array instead of a view, use `numpy.copy`:

```
>>> a = np.array([1, 2, 3, 4])
>>> b = a[:2].copy()
>>> b += 1
>>> print('a = ', a, 'b = ', b)
a = [1 2 3 4] b = [2 3]
```

There are a number of routines to join existing arrays, e.g. `numpy.vstack`, `numpy.hstack`, and `numpy.block`:

```
>>> A = np.ones((2, 2))
>>> B = np.eye(2, 2)
>>> C = np.zeros((2, 2))
>>> D = np.diag((-3, -4))
>>> np.block([[A, B], [C, D]])
array([[ 1.,  1.,  1.,  0.],
       [ 1.,  1.,  0.,  1.],
       [ 0.,  0., -3.,  0.],
       [ 0.,  0.,  0., -4.]])
```

4) Reading Arrays from Disk

This is the most common case of large array creation. NumPy has two standard routines for importing a file with delimited data: `numpy.loadtxt` and `numpy.genfromtxt`.

```
>>> np.loadtxt('simple.csv', delimiter=',', skiprows=1)
array([[0., 0.],
       [1., 1.],
       [2., 4.],
       [3., 9.]])
```

More generic ASCII files can be read using `scipy.io` and `Pandas`.

5) Creating Arrays from Raw Bytes

If the file has a relatively simple format, one can use the NumPy `fromfile()` function and `.tofile()` method to read and write NumPy arrays directly.

6) Use of Special Library Functions

NumPy is the fundamental library for array containers in the Python Scientific Computing stack. Many Python libraries, including `SciPy`, `Pandas`, and `OpenCV`, use NumPy `ndarrays` as the common format for data

exchange. These libraries can create, operate on, and work with NumPy arrays.

Source: NumPy v2.5 Manual, © NumPy Developers.